

Aug 7, 2025

GPT-5 Prompt Migration and Improvement Using the New Optimizer



Raj Pathak (OpenAI), Corwin Cheung (OpenAI)

[View on GitHub](#)[Download raw](#)

The GPT-5 family of models is the smartest models we've released to date, representing a step change in the models' capabilities across the board. GPT-5 is particularly specialized in agentic task performance, coding, and steerability, making it a great fit for everyone from curious users to advanced researchers.

GPT-5 will benefit from all the traditional [prompting best practices](#), but to make optimizations and migrations easier, we are introducing the [GPT-5 Prompt Optimizer](#) in our Playground to help users get started on **improving existing prompts** and **migrating prompts** for GPT-5 and other OpenAI models.

Optimize

Copy

Save

×

Original prompt

Developer message

You are P.A.C.E. (Pickleball Analytics & Coaching Engine), an elite, multidisciplinary performance consultant for advanced and professional pickleball players. Give tips to help someone get better at pickleball. Cover everything that might be useful, like how to hit the ball, move on the court, and win points.

Include:

- How to swing better (forehand, backhand, serve)
- Some basic strategies for winning more
- How to move faster on the court
- Exercises they can try to get stronger
- Mindset tips for staying positive and confident

Try to mention drills and practice routines. Also talk about good equipment. Give general advice that works for anyone.

Instruction to LLM:

You don't need to ask questions first. Just give the advice based on what you know. Keep the tips simple so beginners can understand. Suggest common drills like practicing serves or hitting against a wall. Talk about watching

Request changes (optional)

↑

In this cookbook we will show you how to use the Prompt Optimizer to get spun up quickly to solve your tasks with GPT-5, while demonstrating how prompt optimi

[Ask AI](#)

Loading docs agent...

```
2
3 required = ('OPENAI_API_KEY',)
4 missing = [k for k in required if not os.getenv(k)]
5 print('OPENAI_API_KEY is set!' if not missing else 'Missing
environment variable: ' + ', '.join(missing) + '. Please
set them before running the workflow.')
```

OPENAI_API_KEY is set!



```
## Let's install our required packages
%pip install -r requirements.txt --quiet
```



Loading docs agent...

```
1 baseline_prompt = """  
2 Write Python to solve the task on a MacBook Pro (M4 Max).  
   Keep it fast and lightweight.  
3  
4 - Prefer the standard library; use external packages if  
   they make things simpler.  
5 - Stream input in one pass to keep memory low; reread or  
   cache if that makes the solution clearer.  
6 - Aim for exact results; approximate methods are fine when  
   they don't change the outcome in practice.  
7 - Avoid global state; expose a convenient global like top_k  
   so it's easy to check.  
8 - Keep comments minimal; add brief explanations where  
   helpful.  
9 - Sort results in a natural, human-friendly way; follow  
   strict tie rules when applicable.  
10
```



Loading docs agent...

should callers read globals? Models may implement both, causing side effects that complicate evaluation and reproducibility.

Documentation guidance is similarly split: “keep comments minimal” but “add brief explanations.” Depending on how the model interprets this, you can get under-explained code or prose interleaved with logic, which sometimes leaks outside the required output format.

Finally, we ask for “natural, human-friendly” sorting while also mentioning strict tie rules. These aren’t always the same. The model might pick convenience ordering (e.g., `Counter.most_common`) and drift from the evaluator’s canonical `(-count, token)` sort, especially on ties—leading to subtle correctness misses.

Why this matters: the softened constraints make the prompt feel easy to satisfy, but they create forks in the road. The model may pick different branches across runs—

Loading docs agent...

```
required.
14 - Tokens are ASCII [a-z0-9]+ sequences; treat all other
    characters as separators.
15
16 Output:
17 - Define top_k as a list of (token, count) tuples.
18 - Sort by count desc, then token asc.
19 - Length = min(k, number of unique tokens).
20
21 Notes:
22 - Run as-is with the provided globals; no file or network
    I/O.
23 """
24
25 generate_baseline_topk(
26     model=MODEL,
27     n_runs=N_RUNS,
28     concurrency=CONCURRENCY,
```

Loading docs agent...

model and task. To start let's do just this.

Loading docs agent...

Loading docs agent...

Loading docs agent...

Max).

```
4
5 # Hard requirements
6 - Use only Python stdlib. No approximate algorithms.
7 - Tokenization: ASCII [a-z0-9]+ on the original text; match
  case-insensitively and lowercase tokens individually. Do
  NOT call text.lower() on the full string.
8 - Exact Top-K semantics: sort by count desc, then token
  asc. No reliance on Counter.most_common tie behavior.
9 - Define `top_k` as a list of (token, count) tuples with
  length = min(k, number of unique tokens).
10 - When globals `text` (str) and `k` (int) exist, do not
    reassign them; set `top_k` from those globals. If you
    include a `__main__` demo, guard it to run only when
    globals are absent.
11 - No file I/O, stdin, or network access, except optionally
    printing `top_k` as the last line.
12
```

Loading docs agent...

```
31 from collections import Counter
32 from typing import List, Tuple, Iterable
33
34 _TOKEN = re.compile(r"[a-z0-9]+", flags=re.ASCII |
35 re.IGNORECASE)
36
37 def _tokens(s: str) -> Iterable[str]:
38     # Case-insensitive match; lowercase per token to avoid
39     # copying the whole string
40     for m in _TOKEN.finditer(s):
41         yield m.group(0).lower()
42
43 def top_k_tokens(text: str, k: int) -> List[Tuple[str,
44 int]]:
45     if k <= 0:
46         return []
47     cnt = Counter(_tokens(text))
48     u = len(cnt)
```

Loading docs agent...

```
5 CONCURRENTY = 10
6 OUTPUT_DIR = "results_topk_optimized"
7
8 USER_PROMPT = """
9 Task:
10 Given globals text (str) and k (int), produce the Top-K
    most frequent tokens.
11
12 Tokenization:
13 - Case-insensitive tokenization using an ASCII regex;
    produce lowercase tokens. Whole-string lowercasing is not
    required.
14 - Tokens are ASCII [a-z0-9]+ sequences; treat all other
    characters as separators.
15
16 Output:
17 - Define top_k as a list of (token, count) tuples.
18 - Sort by count desc, then token asc.
```

Loading docs agent...

Adding LLM-as-a-Judge Grading

Along with more quantitative evaluations we can measure the models performance on more qualitative metrics like code quality, and task adherence. We have created a sample prompt for this called `llm_as_judge.txt`.

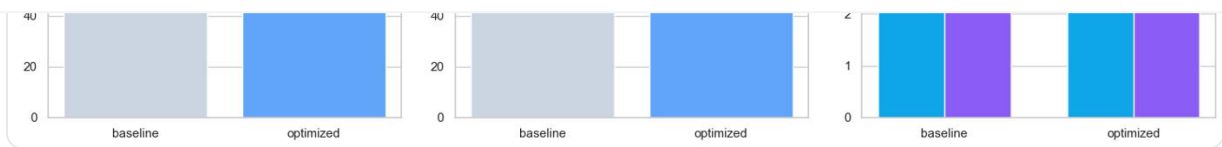
```
from scripts.llm_judge import judge_folder
```

```
1
2 # Run LLM-as-judge for baseline results
3 judge_folder(
4     results_dir="results_topk_baseline",
5     out_dir=None, # auto-map to
6     results_llm_as_judge_baseline
7     model="gpt-5",
```

Loading docs agent...

```
8 fig = rs.render_charts(  
9     quant_baseline=Path("results_topk_baseline")/"run_results_t  
    opk_baseline.csv",  
10    quant_optimized=Path("results_topk_optimized")/"run_results  
    _topk_optimized.csv",  
11    judge_baseline=Path("results_llm_as_judge_baseline")/"judge  
    ment_summary.csv",  
12    judge_optimized=Path("results_llm_as_judge_optimized")/"jud  
    gement_summary.csv",  
13    auto_display=True,  
14    close_after=True,  
15 )  
16 md = rs.build_markdown_summary(
```

Loading docs agent...



Prompt Optimization Results - Coding Tasks

Metric	Baseline	Optimized	Δ (Opt – Base)
Avg Time (s)	7.906	6.977	-0.929
Peak Memory (KB)	3626.3	577.5	-3048.8
Exact (%)	100.0	100.0	0.0
Sorted (%)	100.0	100.0	0.0
LLM Adherence (1-5)	4.40	4.90	+0.50

Loading docs agent...

Context and Retrieval: Simulating a Financial Question Answering

Most production use cases face imperfect queries and noisy context. **FailSafeQA** is an excellent benchmark that deliberately perturbs both the **query** (misspellings, incompleteness, off-domain phrasing) and the **context** (missing, OCR-corrupted, or irrelevant docs) and reports **Robustness**, **Context Grounding**, and **Compliance**—i.e., can the model answer when the signal exists and abstain when it doesn't.

Loading docs agent...

Optimized prompts side by side.

```
1 # Define the Baseline FailSafeQA system prompt here for reuse
2 baseline_prompt_fsqa = (
3     "You are a finance QA assistant. Answer ONLY using the
   provided context.\n"
4     "If the context is missing or irrelevant, politely
   refuse and state that you need the relevant document."
5 )
```



We can use the prompt optimizer once again to construct a new prompt that is more suitable for this use case. Drawing on best practices for long-context question answering, we know that we should remind our answer model to rely on information in the context section and refuse answers to questions if the context is insufficient. By

Loading docs agent...

- 6 3) Robustness to query noise: The user question may contain misspellings, missing words, or non-financial phrasing. Infer intent using the context and answer if the meaning is clear and supported by the context.
- 7 4) OCR noise handling: The context may include OCR artifacts (repeated characters, stray symbols, broken words). Ignore junk characters and reconstruct meaning when the underlying sentence is still recoverable. Do not guess beyond what the context supports.
- 8
- 9 Refusal policy:
- 10 – If [Context] is empty or lacks the information to answer, reply with a brief refusal and guidance. Do NOT attempt a general-knowledge answer.
- 11 – If the question is unrelated to the content of [Context] (out of scope), reply with a brief refusal and guidance. Do NOT speculate.

Loading docs agent...

```
1 import importlib
2 import run_FailSafeQA
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from openai import OpenAI
6
7 # Ensure latest function signature is used after code edits
8 importlib.reload(run_FailSafeQA)
9 run_failsafeqa = run_FailSafeQA.run_failsafeqa
10
11 # Set idx to an integer for a quick single-example
12   comparison; set to None for full run
13   idx = 0 # e.g., 0 for a single datapoint
14
15 #Helper functions:
16 class OpenAIAnswer:
```



Loading docs agent...

```
    "auto"},
41         tools=[],
42     )
43     return resp.output_text
44
45     if idx is not None:
46         # Single example mode (with detailed prompt/response
47         logging)
48         run_failsafeqa(
49             out="results_failsafeqa_baseline.csv",
50             system_prompt=baseline_prompt_fsqa,
51             indices=[idx],
52             log_prompts=True,
53             log_chars=800,
54             log_file="failsafeqa_debug.log",
55         )
56         run_failsafeqa(
57             out="results_failsafeqa_optimized.csv",
```

Loading docs agent...

```
83     base_df =
pd.read_csv("results_failsafeqa_baseline.csv")
84     opt_df =
pd.read_csv("results_failsafeqa_optimized.csv")
85
86     def per_kind_summary(df: pd.DataFrame) -> pd.DataFrame:
87         out = df.groupby("kind").agg(
88             mean_rating=("rating", lambda x:
pd.to_numeric(x, errors="coerce").mean()),
89             compliance_rate=("compliance", lambda x:
pd.to_numeric(x, errors="coerce").fillna(0).mean()),
90             count=("rating", "count"),
91         )
92         return out.round(3)
93
94     base_summary = per_kind_summary(base_df)
95     opt_summary = per_kind_summary(opt_df)
96
```

Loading docs agent...

```
123         return {
124             "mean_rating":
125             float(pd.to_numeric(df["rating"], errors="coerce").mean()),
126             "mean_compliance":
127             float(pd.to_numeric(df["compliance"],
128             errors="coerce").fillna(0).mean()),
129         }
130
131     print("Overall – Baseline:", overall(base_df))
132     print("Overall – Optimized:", overall(opt_df))
```

```
1 from IPython.display import Markdown, display
2
3 def build_markdown_summary_from_metrics(
4     robust_base: float, ground_base: float,
5     robust_opt: float, ground_opt: float,
6     threshold: int = 6,
```



Loading docs agent...

```
33
34     return f"""
35     ## FailSafeQA – Summary
36
37     **Compliance threshold:** ≥ {threshold}
38
39     {table}
40
41     _Source files: _ `{src_base}` · `{src_opt}`
42     """.strip()
43
44     # Usage
45     md = build_markdown_summary_from_metrics(
46         robust_base=0.320, ground_base=0.800,
47         robust_opt=0.540, ground_opt=0.950,
48         threshold=6,
49         src_base="results_failsafeqa.csv",
50         src_opt="results_failsafeqa.csv",
```

Loading docs agent...

Context Grounding (avg across datapoints)	0.800	0.950
+0.150		

Source files: `results\_failsafeqa.csv` ·
`results\_failsafeqa.csv`

GPT-5-mini crushes this task, so even the baseline prompt gets scores of ≥ 4 almost all of the time. However if we compare the percent of perfect scores (6/6) for the judge, we see that the optimize prompt has way significantly more perfect answers when evaluated in the two categories of FailSafeQA answer quality: robustness and context grounding.

Conclusion

Loading docs agent...